Templates — Write It Once, Use It for Any Type

Review worksheet — function & class templates, requirements, non-type parameters, function objects, specialization, CRTP, concepts — builds on Worksheet 06

Name _____ Date _____ Section _____

part 1 One Recipe, Many Types

template_01.cpp

In Worksheet 06 you met two kinds of polymorphism and saw `template` listed under **static** (compile-time). This worksheet looks at templates themselves. Start with the problem they solve: the same algorithm, copied once for each type.

```
int LargerInt(int first, int second) {
    return first > second ? first : second;
}
double LargerDouble(double first, double second) {
    return first > second ? first : second;
}

// One function template replaces both (and every other type, too):
template <typename T>
T Larger(T first, T second) {
    return first > second ? first : second;
}
```

QUESTION 1 — FILL IN THE BLANKS

1. In `template <typename T>`, `T` is a template _____. In `Larger<double>(4, 9.2)`, `double` is a template _____.
2. When the compiler works out `T` from the function's arguments, it is doing template argument _____.
3. The real function the compiler generates for `Larger(3, 7)` is called an _____ of the template. It is generated at _____ time.
4. In `template <class T>`, the word `class` means exactly the same thing as _____. It does **not** restrict `T` to class types.

Word bank argument compile deduction instantiation parameter run specialization typename virtual

QUESTION 2 — WHAT IS T? WHAT PRINTS?

For each call, write the type the compiler uses for `T` and the value printed by `cout << Larger(...)`. If the call does not compile, write **ERROR**.

CALL	T IS…	PRINTS
`(a) Larger(3, 7)`		
`(b) Larger(2.5, 1.5)`		
`(c) Larger(4, 9.2)`		
`(d) Larger<double>(4, 9.2)`		
`(e) Larger('a', 'Z')`		
`(f) Larger(string("apple"), string("Banana"))`		

Call `(c)`: why can't the compiler pick a `T`? Give one way to fix it besides `(d)`.

_____

_____

Templates — Write It Once, Use It for Any Type

Review worksheet — function & class templates, requirements, non-type parameters, function objects, specialization, CRTP, concepts — builds on Worksheet 06

part 2 What a Template Requires

template_02.cpp

A template does not work for *every* type. It works for every type that supports the operations its body uses. Those operations are the template's **requirements**. They aren't written down anywhere (until Part 10). You find them by reading the code.

```
template <typename T>
T Average(const vector<T>& values) {
    T total{};
    for (const T& value : values) {
        total += value;
    }
    return total / values.size();
}
```

QUESTION 3 — LIST THE REQUIREMENTS

List every operation `Average` needs from `T`. (There are at least three. Look at each line.)

QUESTION 4 — WHICH TYPES WORK?

Write the result, or **ERROR** plus the operation that's missing.

`Average(vector<double>{7, 8})`

`Average(vector<int>{7, 8})` (careful)

`Average(vector<string>{"a", "b"})`

For the `string` call, does the compiler's error point at the line in `main()` or at a line *inside* `Average`? Which line?

QUESTION 5 — MAKE IT FIT — OPERATOR OVERLOADING MEETS TEMPLATES

```
class Player {
public:
    Player(const string& name, int score)
        : name_(name), score_(score) {
    }
    int Score() const {
        return score_;
    }
private:
    string name_;
    int score_;
};

Player winner = Larger(Player("Thorin", 85), Player("Malakai", 92));   // ERROR
```

Write the **one function** (outside the class) that makes the last line compile and pick the higher score. Do not change `Larger`.

What does it mean that we made `Player` fit the template, and didn't change the template to fit `Player`?

Templates — Write It Once, Use It for Any Type

Review worksheet — function & class templates, requirements, non-type parameters, function objects, specialization, CRTP, concepts — builds on Worksheet 06

part 3 Class Templates

template_03 & 04

A class template is a recipe for classes. `Box<int>` and `Box<string>` are two separate classes made from one recipe. Everything you know about classes still applies: private data, constructors, `const` methods. In a real project the whole template lives in `Box.h`. Methods can be defined inside the class, or below it like this:

```
template <typename T>
class Box {
public:
    explicit Box(const T& value);
    const T& Get() const;
    T Replace(const T& new_value);   // store new_value, return the OLD value
private:
    T value_;
};
```

QUESTION 6 — WRITE THE DEFINITIONS — BELOW THE CLASS, SAME HEADER

Write all three member functions **outside** the class. Watch the two things every out-of-class template definition needs.

QUESTION 7 — TRUE / FALSE

`Box<int>` and `Box<string>` are the same type, so one can be assigned to the other. _____

`Box<int> b = 5;` compiles. _____

`Box b{2.5};` compiles and creates a `Box<double>`. _____

A member function you never call is never generated, so it can use operations `T` doesn't have. _____

Template member definitions belong in a `.cpp` file, the same as normal classes. _____

In `Box<string>`, the member `value_` is a `string`. _____

Templates — Write It Once, Use It for Any Type

Review worksheet — function & class templates, requirements, non-type parameters, function objects, specialization, CRTP, concepts — builds on Worksheet 06

part 4 Value Parameters and Defaults

template_05.cpp

A template parameter can be a **value** instead of a type. The value must be known at compile time, and it becomes part of the type. A template parameter can also have a **default**, just like a function parameter.

```
template <typename T, size_t Capacity>
class Inventory {
public:
    bool Add(const T& item);
    constexpr size_t MaxSize() const {
        return Capacity;
    }
private:
    array<T, Capacity> items_{};
    size_t count_ = 0;
};

template <typename T = int>
class Counter {
public:
    void Increment() {
        ++value_;
    }
private:
    T value_{};
};
```

QUESTION 8 — COMPILES? Y / N

Judge each line on its own. If it doesn't compile, say why in a few words.

#	CODE	Y / N	WHY NOT?
1	`Inventory<string, 3> backpack;`		
2	`Inventory<int, 4> a; Inventory<int, 8> b; a = b;`		
3	`size_t n = 5; Inventory<int, n> c;`		
4	`constexpr size_t kSlots = 5; Inventory<int, kSlots> d;`		
5	`Counter<> turns;`		
6	`Counter<string> name;`		
7	`Counter<string> name; name.Increment();`		

Lines 6 and 7 together show something important about class templates. What?

QUESTION 9 — DESIGN DECISION

Your `Inventory` could take its capacity as a **template argument** (`Inventory<Item, 10>`) or as a **constructor argument** (`Inventory<Item> bag(10);`). Give one reason to choose each.

Templates — Write It Once, Use It for Any Type

Review worksheet — function & class templates, requirements, non-type parameters, function objects, specialization, CRTP, concepts — builds on Worksheet 06

part 5 A Generic Container

template_06.cpp · builds on W01 stacks

Containers are where templates pay for themselves. A stack works the same way whatever it holds. In Worksheet 01 you wrote a stack of `int`s. Here it is once, for every type. Notice the design: `Stack<T>` **has a** `vector<T>` (composition) and only exposes stack operations.

```
template <typename T>
class Stack {
public:
    void Push(const T& item) {
        items_.push_back(item);
    }
    T Pop() {
        if (items_.empty()) {
            throw underflow_error("Stack::Pop on empty stack");
        }
        T top = items_.back();
        items_.pop_back();
        return top;
    }
    const T& Peek() const {
        return items_.back();
    }
    size_t Size() const {
        return items_.size();
    }
private:
    vector<T> items_;
};
```

QUESTION 10 — TRACE IT

```
Stack<string> s;
s.Push("red");
s.Push("green");
s.Push("blue");
cout << s.Pop() << "\n";    // (a)
s.Push("gold");
cout << s.Peek() << "\n";   // (b)
cout << s.Size() << "\n";   // (c)
cout << s.Pop() << "\n";    // (d)
cout << s.Pop() << "\n";    // (e)
s.Pop();
s.Pop();                    // (f)
```

(a) _____

(b) _____

(c) _____

(d) _____

(e) _____

(f) what happens? _____

QUESTION 11 — WRITE THE METHOD — CONTAINS

Write `bool Contains(const T& item) const` as it would appear inside the class.

What **new** requirement does `Contains` put on `T`? _____

Could you still make a `Stack<Card>` if `Card` doesn't have it? _____

QUESTION 12 — COMPOSITION VS. INHERITANCE

Suppose instead we wrote `class Stack : public vector<T> { ... }`. Name two things a user could then do to a "stack" that break its last-in-first-out rule.

_____

_____

Templates — Write It Once, Use It for Any Type

Review worksheet — function & class templates, requirements, non-type parameters, function objects, specialization, CRTP, concepts — builds on Worksheet 06

part 6 An Interface Without Inheritance

template_07.cpp · compare W06 Part 7

In Worksheet 06, unrelated shapes shared an interface by inheriting from an abstract `Shape`. A template can do something similar **without** a base class: any type with the methods the template calls will fit. This is sometimes called *duck typing*: "if it has `Name()` and `Score()`, it's rankable."

```
template <typename T>
class Leaderboard {
public:
    explicit Leaderboard(size_t max_entries)
        : max_entries_(max_entries) {
    }
    void Submit(const T& entry) {
        entries_.push_back(entry);
        sort(entries_.begin(), entries_.end(), [](const T& left, const T& right) {
            return left.Score() > right.Score();
        });
        if (entries_.size() > max_entries_) {
            entries_.pop_back();
        }
    }
private:
    vector<T> entries_;
    size_t max_entries_;
};

class Player { /* string Name() const;   int Score() const;    */ };
class Team   { /* string Name() const;   double Score() const; */ };   // win %, a double
class Monster { /* string Species() const;   int HitPoints() const; */ };
```

QUESTION 13 — TRACE THE BOARD

`Leaderboard<Player> board(3);` Write the board, top to bottom, after each `Submit`.

SUBMIT	BOARD AFTER (NAME AND SCORE)
`Player("Thorin", 85)`	
`Player("Malakai", 92)`	
`Player("Jordan", 77)`	
`Player("Rhea", 99)`	
`Player("Vex", 80)`	

QUESTION 14 — WHERE'S THE ERROR?

```
Leaderboard<Team> teams(2);                 // (1)
Leaderboard<Monster> monsters(5);           // (2)
monsters.Submit(Monster("Troll", 40));      // (3)
```

Which numbered line(s) fail to compile? _____ Why does line (2) on its own *not* fail? Why is line (1) fine even though `Team::Score()` returns a `double`?

Templates — Write It Once, Use It for Any Type

Review worksheet — function & class templates, requirements, non-type parameters, function objects, specialization, CRTP, concepts — builds on Worksheet 06

part 6 An Interface Without Inheritance (continued)

QUESTION 15 — WRITE AN ADAPTER — YOU MAY NOT EDIT MONSTER OR LEADERBOARD

`Monster` comes from a library you can't change. Write a small class `RankedMonster` that **holds** a `Monster` and gives it the interface `Leaderboard` needs. Use the species as the name and the hit points as the score.

```
class RankedMonster {

};

Leaderboard<RankedMonster> monsters(5);
monsters.Submit(RankedMonster(Monster("Troll", 40)));
```

QUESTION 16 — TEMPLATE VS. ABSTRACT BASE CLASS

Someone rewrites `Leaderboard` to store `Rankable*`, where `Rankable` is an abstract base with pure virtual `Name()` and `Score()`.

1. What must `Player`, `Team`, and `Monster` now do that they didn't before? _____
2. `Player::Score()` returns `int`, `Team::Score()` returns `double`. What return type must the pure virtual `Score()` have? What changes in the derived classes? _____
3. Name one thing the `Rankable*` version can do that `Leaderboard<T>` can't.
   _____

QUESTION 17 — THE SAME IDEA, ONE LINE EACH

	TEMPLATE VERSION	ABSTRACT BASE VERSION
Where the interface is written down		
When a missing method is caught		
Can one board mix Players and Teams?		

Templates — Write It Once, Use It for Any Type

Review worksheet — function & class templates, requirements, non-type parameters, function objects, specialization, CRTP, concepts — builds on Worksheet 06

part 7 Passing Behavior In

template_08.cpp

A template parameter can stand for *something you can call*: a function, a lambda, or a **function object** (a class with `operator()`). This lets the caller decide part of the behavior. It's the Strategy pattern, chosen at compile time. The standard library does this everywhere: `sort`, `map`, `set`.

```
template <typename T, typename Predicate>
vector<T> Filter(const vector<T>& items, Predicate keep) {
    vector<T> result;
    for (const T& item : items) {
        if (keep(item)) {
            result.push_back(item);
        }
    }
    return result;
}

bool IsEven(int value) {
    return value % 2 == 0;
}

class AtLeast {
public:
    explicit AtLeast(int threshold)
        : threshold_(threshold) {
    }
    bool operator()(int value) const {
        return value >= threshold_;
    }
private:
    int threshold_;
};
```

QUESTION 18 — PREDICT THE RESULT

Use `vector<int> scores = {72, 95, 88, 41, 100, 63};`

EXPRESSION	RESULT
`Filter(scores, IsEven)`	
`Filter(scores, AtLeast(80))`	
`Filter(scores, [](int s) { return s < 70; })`	

QUESTION 19 — WRITE A FUNCTION OBJECT — BETWEEN

Write a class `Between` so that `Filter(scores, Between(60, 90))` keeps scores from 60 to 90, inclusive. Model it on `AtLeast`.

Why can't a plain function like `IsEven` do this job as neatly? What does the class have that the function doesn't?

Templates — Write It Once, Use It for Any Type

Review worksheet — function & class templates, requirements, non-type parameters, function objects, specialization, CRTP, concepts — builds on Worksheet 06

part 7 Passing Behavior In (continued)

A class template can take a behavior too. `SortedList` doesn't hard-code what "in order" means. The second template parameter is the ordering strategy, and it defaults to the standard library's `less<T>` (which uses `<`).

```
template <typename T, typename Compare = less<T>>
class SortedList {
public:
    void Insert(const T& item) {                        // keeps items_ in order
        auto position = items_.begin();
        while (position != items_.end() && compare_(*position, item)) {
            ++position;
        }
        items_.insert(position, item);
    }
private:
    vector<T> items_;
    Compare compare_;
};

struct ByLength {
    bool operator()(const string& left, const string& right) const {
        return left.size() < right.size();
    }
};
```

QUESTION 20 — PREDICT THE ORDER

Trace `Insert` carefully. It stops at the first item that does **not** belong before the new one.

LIST	CONTENTS, FRONT TO BACK
`SortedList<int, greater<int>>` after inserting `5, 1, 4, 1`	
`SortedList<string, ByLength>` after inserting `"dragon", "elf", "wizard", "orc"` (careful with ties)	

QUESTION 21 — SHORT ANSWER

1. Are `SortedList<int>` and `SortedList<int, greater<int>>` the same type? _____ Why?

   _____

2. `SortedList<int>` doesn't say what `Compare` is. What does it use, and how does it sort?

   _____

3. Another design gives `SortedList` a `Comparator*` to an abstract class with a virtual `Compare()`. When would that be the better choice?

   _____

Templates — Write It Once, Use It for Any Type

Review worksheet — function & class templates, requirements, non-type parameters, function objects, specialization, CRTP, concepts — builds on Worksheet 06

part 8 Specialization

template_09.cpp

A **full specialization** (`template <>`) replaces a template for one exact type. A **partial specialization** replaces it for a family of types, like every `vector<T>`. The compiler picks the *most specialized* match.

```
template <typename T>                       // primary: anything that supports <<
class Formatter {
public:
    static string Format(const T& value) {
        ostringstream out;
        out << value;
        return out.str();
    }
};
template <>                                 // full specialization: exactly bool
class Formatter<bool> {
public:
    static string Format(const bool& value) {
        return value ? "yes" : "no";
    }
};
template <>                                 // full specialization: exactly string
class Formatter<string> {
public:
    static string Format(const string& value) {
        return "\"" + value + "\"";
    }
};
template <typename T>                       // partial specialization: any vector
class Formatter<vector<T>> {
public:
    static string Format(const vector<T>& values) {
        string result = "[";
        for (size_t i = 0; i < values.size(); ++i) {
            if (i > 0) {
                result += ", ";
            }
            result += Formatter<T>::Format(values[i]);
        }
        return result + "]";
    }
};
template <typename T>
void Show(const string& label, const T& value) {
    cout << label << " = " << Formatter<T>::Format(value) << "\n";
}
```

QUESTION 22 — PREDICT THE OUTPUT

CALL	PRINTS
`Show("level", 7);`	
`Show("alive", true);`	
`Show("name", string("Thorin"));`	
`Show("flags", vector<bool>{true, false});`	
`Show("party", vector<string>{"Thorin", "Rhea"});`	
`Show("grid", vector<vector<int>>{{1, 2}, {3}});`	
`Show("name", "Thorin");` (no `string(...)`)	

Templates — Write It Once, Use It for Any Type

Review worksheet — function & class templates, requirements, non-type parameters, function objects, specialization, CRTP, concepts — builds on Worksheet 06

part 8 Specialization (continued)

QUESTION 23 — SHORT ANSWER

1. Explain the last row of Question 22. What is `T`, and which `Formatter` runs?

2. The `grid` row worked even though nobody wrote a `Formatter` for `vector<vector<int>>`. Explain how.

3. Delete `Formatter<bool>`. What does `Show("alive", true)` print now, and which version handles it?

Function templates are different. You can fully specialize a function template, but you *cannot partially specialize* one. To give a function template special behavior for a family of types, you **overload** it instead.

```
template <typename T>
T Total(const vector<T>& values) {          // (1) general version
    T sum{};
    for (const T& value : values) {
        sum += value;
    }
    return sum;
}

int Total(const vector<bool>& values) {      // (2) an overload, not a specialization
    int count = 0;
    for (bool value : values) {
        if (value) {
            ++count;
        }
    }
    return count;
}
```

QUESTION 24 — WHICH VERSION RUNS?

CALL	(1) OR (2)?	RETURNS
`Total(vector<int>{5, 10, 15})`		
`Total(vector<double>{1.5, 2.25})`		
`Total(vector<bool>{true, true, false, true})`		
`Total(vector<string>{"ab", "cd"})`		

Without overload (2), what would `Total(vector<bool>{true, true, false, true})` try to return? Why is that not useful?

Templates — Write It Once, Use It for Any Type

Review worksheet — function & class templates, requirements, non-type parameters, function objects, specialization, CRTP, concepts — builds on Worksheet 06

part 9 Templates and Inheritance Together

template_10 & 11 · builds on W06

Use **inheritance** when different types must be mixed together and chosen *at run time*. Use **templates** when the same code should work for many types. Strong designs use both: `vector<unique_ptr<Character>>` is a template holding a polymorphic hierarchy.

QUESTION 25 — PICK THE TOOL — TEMPLATE / VIRTUAL / BOTH

- A `Matrix` class that should work for `int`, `double`, and your W04 `Fraction` _____
- A battle loop that holds Warriors, Wizards, and Rogues in one container and calls `attack()` _____
- A `Queue` used for print jobs in one program and game events in another _____
- A drawing app that reads shape names from a file and creates the matching shape objects _____
- One `Spawn<Derived>(name)` function that builds any `Character` subclass and returns a `unique_ptr<Character>` _____

A base class can itself be a template. In the pattern below (called **CRTP**), each class passes *itself* to its base. `Comparable<Coin>` writes all the comparison operators in terms of one method, `LessThan()`. `Counted<Coin>` counts live objects with a `static` member.

```
template <typename Derived>
class Comparable {
public:
    friend bool operator<(const Derived& left, const Derived& right) {
        return left.LessThan(right);
    }
    friend bool operator!=(const Derived& left, const Derived& right) {
        return left.LessThan(right) || right.LessThan(left);
    }
    // ... plus >, >=, <=, == written the same way
};

template <typename Derived>
class Counted {
public:
    Counted() {
        ++alive_;
    }
    Counted(const Counted&) {
        ++alive_;
    }
    ~Counted() {
        --alive_;
    }
    static int Alive() {
        return alive_;
    }
private:
    static inline int alive_ = 0;
};

class Coin : public Comparable<Coin>, public Counted<Coin> {
public:
    explicit Coin(int cents)
        : cents_(cents) {
    }
    bool LessThan(const Coin& other) const {
        return cents_ < other.cents_;
    }
private:
    int cents_;
};
class Gem : public Comparable<Gem>, public Counted<Gem> { /* similar, compares carats */ };
```

Templates — Write It Once, Use It for Any Type

Review worksheet — function & class templates, requirements, non-type parameters, function objects, specialization, CRTP, concepts — builds on Worksheet 06

part 9 Templates and Inheritance Together (continued)

QUESTION 26 — PREDICT THE OUTPUT

```
Coin dime(10);
Coin quarter(25);
cout << (dime < quarter);      // (a)
cout << (dime >= quarter);     // (b)
cout << (dime != Coin(10));    // (c)
{
    Coin penny(1);
    Coin copy = penny;
    cout << Coin::Alive();     // (d)
}
cout << Coin::Alive();         // (e)
Gem ruby(3);
cout << Counted<Gem>::Alive(); // (f)
```

(a) _____

(b) _____

(c) _____

(d) _____

(e) _____

(f) _____

(`cout` prints `bool` as `1` or `0`.)

QUESTION 27 — SHORT ANSWER

1. Why does `Coin` get its own count, separate from `Gem`, when both inherit the same `alive_` code?

   _____

   _____

2. Does `dime < ruby` compile? _____ Why or why not?

   _____

3. Why couldn't `Comparable` be an ordinary base class with `virtual bool LessThan(const ??? & other)`? What would you write for `???`, and what goes wrong?

   _____

   _____

Templates — Write It Once, Use It for Any Type

Review worksheet — function & class templates, requirements, non-type parameters, function objects, specialization, CRTP, concepts — builds on Worksheet 06

part 10 Concepts: Writing the Requirements Down

template_12.cpp · C++20

In Part 6, `Leaderboard`'s requirements were hidden in its code, and the error for `Monster` showed up deep inside the template. A **concept** turns the requirements into code the compiler checks *where the template is used*. It works a lot like an abstract base class's list of pure virtuals, but nothing has to inherit from anything.

```
template <typename T>
concept Scorable = requires(const T& item) {
    { item.Name() } -> convertible_to<string>;
    { item.Score() } -> totally_ordered;     // supports < > <= >= == !=
};

template <Scorable T>
class Leaderboard { /* same as Part 6 */ };
```

QUESTION 28 — DOES IT SATISFY SCORABLE? Y / N

TYPE	Y / N	IF N, WHY?
`struct Player { string Name() const; int Score() const; };`		
`struct Monster { string Species() const; int HitPoints() const; };`		
`int`		
`struct Ghost { string Name() const; string Score() const; };`		
`struct Bot { int Name() const; int Score() const; };`		
`struct Drone { string Name() const; int Score(); };`		

`Ghost`'s "score" is a `string`. What does that tell you about what a concept can and can't check?

QUESTION 29 — WRITE A CONCEPT — PRINTABLE

Write a concept `Printable` that is satisfied when `cout << item` compiles for a `const T& item`. Then write the first line of a function template `PrintAll` that only accepts a `vector` of `Printable` things.

Templates — Write It Once, Use It for Any Type

Review worksheet — function & class templates, requirements, non-type parameters, function objects, specialization, CRTP, concepts — builds on Worksheet 06

part 11 Find the Error

four traps

CASE A — THREE FILES

```
// Box.h  -- class declaration only
// Box.cpp
template <typename T>
const T& Box<T>::Get() const {
    return value_;
}
// main.cpp
#include "Box.h"
Box<int> b(3);
cout << b.Get();
```

Each `.cpp` compiles. What fails, and at which step (compile or link)? What's the fix?

CASE B — COMPILES? Y / N

```
template <typename T>
class Box {
public:
    const T& Get() const;
private:
    T value_;
};

const T& Box::Get() const {
    return value_;
}
```

Find **two** mistakes in the definition and fix them.

CASE C — COMPILES? Y / N

```
int slots;
cout << "How many slots? ";
cin >> slots;
Inventory<string, slots> bag;
```

What's wrong? If the user really must choose the size, how should `Inventory` be redesigned?

CASE D — COMPILES? Y / N

```
Stack s;
s.Push(5);
s.Push(10);
```

Why can't the compiler work out the type here, when `Box b{2.5};` worked fine?

QUESTION 30 — YOUR OWN DESIGN — P01 CARD GAME

Think about your P01 card game. Name **one** class that would be a good template (what is `T`?) and **one** part that should stay a virtual hierarchy. Give a reason for each.

Templates — Write It Once, Use It for Any Type

Review worksheet — function & class templates, requirements, non-type parameters, function objects, specialization, CRTP, concepts — builds on Worksheet 06

cheat sheet Templates at a Glance

CONCEPT	TYPICAL SYNTAX	PURPOSE
Function template	`template <typename T> T Larger(T a, T b)`	One algorithm for many types
Explicit argument	`Larger<double>(4, 9.2)`	Choose `T` yourself when deduction can't
Class template	`template <typename T> class Box`	One class design, many stored types
Out-of-class member	`template <typename T> const T& Box<T>::Get() const`	Define methods below the class, **in the header**
CTAD	`Box b{2.5};`	Compiler deduces `Box<double>` from the constructor
Non-type parameter	`template <typename T, size_t N>`	A compile-time value that becomes part of the type
Default argument	`template <typename T = int>` · `Counter<>`	Use a type unless told otherwise
Function object	`bool operator()(int v) const`	Behavior with its own data, passed as a template argument
Full specialization	`template <> class Formatter<bool>`	Different code for one exact type
Partial specialization	`template <typename T> class Formatter<vector<T>>`	Different code for a family of types (classes only)
CRTP mixin	`class Coin : public Comparable<Coin>`	Reuse behavior across classes, no `virtual`
Concept	`template <Scorable T>`	State the requirements; errors appear at the use site

recap Static vs. Dynamic Polymorphism

W06 meets W07

	TEMPLATES (STATIC)	VIRTUAL FUNCTIONS (DYNAMIC)
Decided	At compile time	At run time
Shared base class?	Not needed. The type just needs the right operations.	Required
Mixed types in one container?	No. A `vector<T>` holds one `T`.	Yes, through `unique_ptr<Base>`
Interface written as	A concept (or just the code, before C++20)	An abstract base class
Cost	More compiled code (one copy per type)	A small lookup on each virtual call

TAKEAWAY — ONE QUESTION TO ASK

Does the behavior stay the same while the type changes? Use a template. **Does the behavior change depending on which object you have at run time?** Use a virtual function. Often a good design uses both.